

Profesor:	David Aparicio Escribano	Grupo	81
Alumno/a:	Marta Vindel Navas	NIA:	100522119
Alumno/a:	Paula Martínez Lechón	NIA:	100522137
Alumno/a:	Xiya Ye	NIA:	100522159

1. Introducción

Esta práctica consiste en familiarizarse mejor con los elementos operativos que puede tener una base de datos. Partimos de unas tablas con información de la práctica 1, más en concreto, la solución a esta. La idea es tratar en distintos apartados todo lo que se ha ido aprendiendo durante el curso sobre manejo de bases de datos.

Los objetivos son realizar las consultas que se solicitan, así como implementar procedimientos en un paquete para hacer más fácil y sin errores la inserción de préstamos, reservas y devoluciones, vistas para poder mostrar las partes que nos interesan y disparadores para asegurar el fallo o la ejecución necesaria en partes que no se veían cubiertas por la semántica.

Además, como objetivo secundario perseguimos el aprendizaje de estos mediante la ejecución de estos a la hora de realizar la práctica.

En cuanto a la estructura, este documento está dividido en 6 apartados:

- Una breve introducción para saber que se va a encontrar en el documento
- Dos consultas sobre las tablas proporcionadas
- Tres procedimientos relacionados con los préstamos, las reservas y las devoluciones.
- Vistas, tanto de lectura como operables, que a su vez incluyen disparadores
- Disparadores
- Una conclusión con los problemas encontrados, las soluciones y cómo creemos que se nos ha dado la implementación. Además, también se exponen nuestras sensaciones ante las dos prácticas

2. Consultas

Para cada consulta se debe incluir una subsección, en la que se describirán:

- a) su diseño en álgebra relacional
- b) su implementación en SQL
- c) las pruebas realizadas para demostrar que funciona correctamente

Consulta 1: BoreBooks

La consulta que se va a realizar esperamos que muestre libros con ediciones en, al menos, tres idiomas (language) distintos, de los que nunca se haya prestado ninguna copia.

Consideramos que sólo se seleccionarán los libros en el caso de que varíe el idioma en el atributo language para las distintas ediciones. No consideraremos las variaciones de idioma de una misma edición (alter languages), puesto que entendemos por el enunciado que se pretenden encontrar tres ediciones mínimo con idiomas diferentes.

- a) Diseño en álgebra relacional

Para la implementación de esta consulta, hemos hecho una intersección entre los libros con al menos tres idiomas y los libros de los que nunca se han prestado.

Para los libros con al menos tres idiomas, en la primera consulta, hacemos una consulta del título, autor y del idioma de la tabla de Editions, para así en la siguiente proyección podamos contar el número de libros con diferentes idiomas después de agrupar por título y autor. Así, podemos seleccionar directamente usando la selección, escogiendo sólo los que tengan al menos 3 idiomas.

Para los libros no prestados, primero hemos proyectado la signatura de la tabla Loans, con esto, podemos sacar las ediciones que se hayan prestado, se ha usado una combinación natural de la tabla Copies con la consulta anterior. Por otro lado, hacemos una consulta de todas las ediciones en la tabla Editions, y proyectamos los atributos que nos interesan, que son el título, autor y el isbn. Teniendo esto, podemos hacer un left join de las dos consultas anteriores en función del isbn, proyectamos el título, autor, isbn máximo de las ediciones que se han prestado como una variable prestado. Para así poder proyectar el título y autor haciendo una selección donde el prestado sea nulo. Esto nos sacará todos los libros que no se hayan prestado.

Con estas dos consultas, hemos podido conseguir la consulta final con una intersección de ambas.

```

1) Selección-libros-idioma = TTtitle,author,language (Editions)

Conteo-libros-autor = TTtitle,author,count('x') as num
                      G title,author (Selección-libros-idioma)

Libros-idiomas = TTtitle,author (σ num ≥ 3 (Conteo-libros-autor))

Libros-loans = TTSignature (Loans)

Ediciones-prestadas = TTIbn (Copies * signature Libros-loans)

todas-ediciones = TTtitle,author, isbn (Editions)

Prestamos-ediciones = TTtitle,author,max(Ediciones-prestadas.isbn) as prestado
                      G title,author (todas-ediciones <= Ediciones-prestadas.isbn Ediciones-prestadas)

Libros-Noprestados = TTtitle,author (σ prestado is null (Prestamos-ediciones))

Solución = Libros-idiomas n Libros-Noprestados
  
```

b) Implementación en sql:

Se realizan distintas consultas que se ven recogidas mediante alias en un with. Estas consultas van dependiendo unas de otras para llegar al resultado buscado. La última consulta se realiza fuera del with y es la que mostrará el resultado esperado en pantalla. La descripción de cada consulta se encuentra en el subapartado anterior.

Código en SQL:

with

```

SELECCION_LIBROS_IDIOMA as (select distinct title, author, language from Editions),
CONTEO_LIBROS_AUTOR as (select distinct title, author, count('x') as num from
SELECCION_LIBROS_IDIOMA group by (title, author)),
LIBROS_IDIOMAS as (select distinct title, author from CONTEO_LIBROS_AUTOR where
num >=3),
LIBROS_LOANS as (select distinct signature from loans),
EDICIONES_PRESTADOS as (select distinct ISBN from copies natural join
LIBROS_LOANS),
TODAS_EDICIONES as (select distinct title, author, ISBN from Editions),
PRESTAMOS_EDICIONES as (select title, author, max(EDICIONES_PRESTADOS.isbn) as
prestado from TODAS_EDICIONES left join EDICIONES_PRESTADOS on
TODAS_EDICIONES.isbn = EDICIONES_PRESTADOS.isbn group by (title, author)),
LIBROS_NO_PRESTADOS as (select distinct title, author from PRESTAMOS_EDICIONES
where prestado IS NULL)
select distinct * from LIBROS_IDIOMAS INTERSECT select distinct * from
LIBROS_NO_PRESTADOS;
  
```

La consulta devuelve lo siguiente por pantalla (últimas filas):

TITLE	AUTHOR
TÃ pies	TÃ pies, Antoni, (1923-2012)
Valencia	Navarro Lloret, David
Valores de futuro :: programa educativo, 2012-2013.	Ullmo, Pierre-Antoine
Wado-Ryu karate	Åtsuka, Hironori, (1892-1982)
Weng Chun Kung Fu	Hoffmann, Andreas
60 rows selected.	

c) Batería de pruebas

Para comprobar que la consulta se ha realizado correctamente y muestra los resultados esperados por pantalla, realizamos las siguientes pruebas.

- Insertamos un nuevo libro

Para esta prueba insertaremos un nuevo libro en la tabla **books** y tres ediciones distintas de este en la tabla **editions**. Así mismo crearemos copias de estas ediciones en la tabla **copies**, que no se insertarán en la tabla **loans** para comprobar que se muestra una fila más en la consulta con el título y autor introducidos en la creación. Las inserciones a realizar son las siguientes:

- 1) insert into books(title, author) values('Trono de Cristal', 'Maas, Sarah J');
- 2) insert into editions(isbn, title, author, language, national_lib_id) values('978-84-234-0301-5', 'Trono de Cristal', 'Maas, Sarah J', 'ingles', 'a5327301');
- 3) insert into editions(isbn, title, author, language, national_lib_id) values('978-84-234-0301-6', 'Trono de Cristal', 'Maas, Sarah J', 'español', 'a5327302');
- 4) insert into editions(isbn, title, author, language, national_lib_id) values('978-84-234-0301-7', 'Trono de Cristal', 'Maas, Sarah J', 'italiano', 'a5327303');
- 5) insert into copies(signature, isbn, condition) values('SN301', '978-84-234-0301-5', 'N');
- 6) insert into copies(signature, isbn, condition) values('SN302', '978-84-234-0301-6', 'G');
- 7) insert into copies(signature, isbn, condition) values('SN303', '978-84-234-0301-7', 'G');

El resultado obtenido:

TITLE	AUTHOR
Trono de Cristal	Maas, Sarah J
TÃ pies	TÃ pies, Antoni, (1923-2012)
Valencia	Navarro Lloret, David
Valores de futuro :: programa educativo, 2012-2013.	Ullmo, Pierre-Antoine
Wado-Ryu karate	Åtsuka, Hironori, (1892-1982)
Weng Chun Kung Fu	Hoffmann, Andreas
61 rows selected.	

Efectivamente se muestra una fila más que contiene los datos esperados.

- Borramos una edición que cumpliera la condición

Para esta prueba borraremos una edición o varias de un libro que cumpliera la condición para que ya no la satisfaga y las filas mostradas se reduzcan en 1, es decir, a 60 filas. Los borrados a realizar son los siguientes:

- 1) delete from copies where signature = 'SI239' ;
- 2) delete from copies where signature = 'SI101';
- 3) delete from editions where isbn = '978-3-86836-183-4';
- 4) delete from editions where isbn = '978-84-15407-05-8';

El resultado obtenido:

TITLE	AUTHOR
Trono de Cristal	Maas, Sarah J
TÀ pies	TÀ pies, Antoni, (1923-2012)
Valencia	Navarro Lloret, David
Valores de futuro :: programa educativo, 2012-2013.	Ullmo, Pierre-Antoine
Wado-Ryu karate	Åetsuka, Hironori, (1892-1982)

60 rows selected.

Efectivamente se muestra una fila menos y ya no aparece el autor Andreas Hoffmann con el libro Weng Chun Kung Fu.

Consulta 2: Informe de Empleados:

La consulta que se va a realizar esperamos que muestre para cada conductor, su nombre completo, edad, antigüedad de contrato (años completos), años activo (años con al menos un día en carretera), número medio de paradas por año activo, número medio de préstamos por año activo, y porcentaje de préstamos no devueltos en relación a los que ha prestado él.

a) Diseño en álgebra relacional:

En la primera consulta, proyectamos el pasaporte y extraemos solamente el año de los días de trabajo y lo llamamos como activo, esto lo sacamos de la tabla Assign_drv.

Para la consulta de num_años_activo contamos el número de años activo de cada conductor agrupando por el pasaporte y usando count('x').

En la consulta de num_paradas pasamos a la tabla Services y agrupamos por pasaporte para así proyectar el pasaporte y poder contar el número de paradas con count('x').

Con el número de paradas y años activos, podemos hacer una consulta para calcular el número medio de paradas por año activo, para eso, hemos hecho una combinación natural de las consultas que calculan los años activo y el número de paradas, y así poder proyectar el cálculo de las paradas medias.

En la consulta de num_prestamos hemos seleccionado todos los préstamos activos de la tabla Loans, y hemos proyectado todos los atributos que nos interesan, teniendo en cuenta que hemos llamado a stopdate como taskdate, para después al hacer combinaciones no tener problemas.

Para la consulta de los préstamos del bibusero hecho hecho una combinación natural de la tabla Services con la consulta anterior agrupando por pasaporte, para así contar el número de préstamos que ha realizado cada bibusero.

Teniendo el número de préstamos, podemos hacer un left join de las consultas que contienen el número de préstamos y años activo para así calcular la media de préstamos de cada bibusero. Hemos tenido en cuenta que perdemos valores cuando son nulos, por lo que hemos usado la función nvl para que asignen los valores nulos un 0, esto lo hemos hecho tanto para los préstamos como para la media.

Para la consulta de prestamos_no_devueltos hemos hecho una selección de todas las filas donde tiene el atributo devolución nula de la consulta de num_prestamos y hemos proyectado los datos que nos interesan.

En la siguiente consulta, contamos los préstamos no devueltos con una combinación natural de la tabla Services y la consulta de prestamos_no_devueltos, con esto, lo agrupamos por pasaporte y hacemos el conteo de los no devueltos.

Finalmente, calculamos el porcentaje de los libros no devueltos con un left join con la consulta anterior, y como hemos ido arrastrando todos los datos que nos interesan, que son el pasaporte, los años activo, la media de paradas, la media de préstamos y el porcentaje de los no devueltos, proyectamos todos estos datos.

```

Datos.inmediatos = Π fullname,
                    floor((sysdate-birthdate)/365.25) as Edad,
                    floor((sysdate-contr.start)/365.25) as Antigüedad (Drivers)

años activo {
  Años = Π passport, extract(year from taskdate) as activo (Assign.Dn)
  Num.Años.Activos = Π passport, count(*) as años.activo ⋈ passport C Años }

paradas/años {
  Num.Paradas = Π passport, count(*) as paradas ⋈ passport C Services
  Paradas.media = Π passport, años.activo,
                  paradas/años.activo as media.paradas ( Num.Años.Activo * Num.Paradas )

préstamos/años {
  Num.Prestamos = Π signature, user.id, stopdate as taskdate, town, province, devolución ( ⋈ type: 'L' C Loans )
  Prestamos.bibusero = Π passport, count(*) as prestamos ⋈ passport C Services * Num.Prestamos
  Prestamos.media = Π passport, años.activo, media.paradas, nvl(prestamos, 0) as prestamos,
                    nvl(prestamos/años.activo, 0) as media.prestamos ( Paradas.media ⋈ passport Prestamos.bibusero )

no devueltos {
  Rest.No.Devueltos = Π signature, user.id, taskdate, town, province ( ⋈ devolución is Null C Num.Prestamos )
  Conteo.No.Devueltos = Π passport, count(*) as no.devueltos ⋈ passport C Services * Rest.No.Devueltos
  No.Devueltos.Porcentaje = Π passport, años.activo, media.paradas, prestamos, media.prestamos,
                             nvl((no.devueltos/prestamos)*100, 0) as porcentaje.no.devueltos
                             ( Prestamos.media ⋈ passport Conteo.No.Devueltos )

SOLUCION = Π nombre.completo, edad, antigüedad, años.activo, media.paradas,
            media.prestamos, porcentaje.no.devueltos ( Datos.inmediatos * No.Devueltos.Porcentaje )
  
```


b) Implementación en sql:

Se realizan distintas consultas que se ven recogidas mediante alias en un with. Estas consultas van dependiendo unas de otras para llegar al resultado buscado. La última consulta se realiza fuera del with y es la que mostrará el resultado esperado en pantalla. La descripción de cada consulta se encuentra en el subapartado anterior.

Código en SQL:

with

DATOS_INMEDIATOS as (select distinct passport, fullname as nombre_completo, floor((sysdate-birthdate)/365.25) as edad, floor ((sysdate - cont_start)/365.25) as antigüedad from Drivers),

AÑOS as (select distinct passport, extract(year from taskdate) as activo from Assign_Drv),
NUM_AÑOS_ACTIVO as (select distinct passport, count('x') as años_activo from AÑOS group by passport),

NUM_PARADAS as (select distinct passport, count('x') as paradas from Services group by passport),

PARADAS_MEDIA as (select distinct passport, años_activo, paradas/años_activo as media_paradas from NUM_AÑOS_ACTIVO natural join NUM_PARADAS),

NUM_PRESTAMOS as (select distinct signature, user_id, stopdate as taskdate, town, province, devolucion from loans where type = 'L'),

PRESTAMOS_BIBUSERO as (select distinct passport, count('x') as prestamos from services natural join NUM_PRESTAMOS group by passport),

PRESTAMOS_MEDIA as (select distinct passport, años_activo, media_paradas, nvl(prestamos, 0) as prestamos,

nvl(prestamos/años_activo, 0) as media_prestamos from PARADAS_MEDIA left join PRESTAMOS_BIBUSERO using (passport)),

PRESTAMOS_NO_DEVUELTOS as (select distinct * from NUM_PRESTAMOS where devolucion is null),

CONTEO_NO_DEVUELTOS as (select distinct passport, count('x') as no_devueltos from services natural join PRESTAMOS_NO_DEVUELTOS group by passport),

NO_DEVUELTOS_PORCENTAJE as (select distinct passport, años_activo, media_paradas, prestamos, media_prestamos,

nvl((no_devueltos/prestamos)*100, 0) as porcentaje_no_devueltos from PRESTAMOS_MEDIA left join CONTEO_NO_DEVUELTOS using(passport)),

SOLUCION as (select distinct nombre_completo, edad, antigüedad, años_activo, media_paradas, media_prestamos,

porcentaje_no_devueltos from DATOS_INMEDIATOS natural join NO_DEVUELTOS_PORCENTAJE)

select * from Solucion;

La consulta devuelve lo siguiente por pantalla:

NOMBRE_COMPLETO	EDAD	ANTIGÜEDAD	AÑOS_ACTIVO	MEDIA_PARADAS	MEDIA_PRESTAMOS	PORCENTAJE_NO_DEVUELTOS
Maria Pensamiento Calma Grata	42	7	1	110	1788	
Victoria Rojo Blanco	38	2	1	109	1664	
Fuencisla Maja San Frutos	33	5	1	108	2599	
Eulalia Puig Castilla	39	4	1	109	2559	
Octavio Sansegundo Quinto	43	6	1	109	1769	
César Pareja Feliz	55	3	1	55	0	
Zacarias Rico Morcón	42	7	1	108	2136	
Carlos Tercero Madrid	45	2	1	110	1146	
Frutos Campo Dorado	41	2	1	111	2678	
María José Chal Abregado	44	2	1	110	1799	
Amaia Itsasoko Gaztelua	35	4	1	109	1797	
Celso Hórreo Madera	44	7	1	108	2287	
José Pepe Perez Quitasol	43	2	2	55.5	743	

13 rows selected.

c) Batería de pruebas

Para comprobar que la consulta se ha realizado correctamente y muestra los resultados esperados por pantalla, realizamos las siguientes pruebas.

- Insertamos un año trabajado

Para esta prueba insertaremos una nueva taskdate a **assign_drv** de un año distinto al almacenado para comprobar si el número de años activos aumenta en 1. La inserción a realizar es las siguientes:

```
insert into assign_drv values('ESP>>101010110110', '04-APR-23', 'IC-04');
```

El resultado obtenido:

NOMBRE_COMPLETO	EDAD	ANTIGÜEDAD	AÑOS_ACTIVO	MEDIA_PARADAS	MEDIA_PRESTAMOS	PORCENTAJE_NO_DEVUELTOS
Maria Pensamiento Calma Grata	42	7	1	110	1788	
Victoria Rojo Blanco	38	2	1	109	1664	
Fuencisla Maja San Frutos	33	5	1	108	2599	
Eulalia Puig Castilla	39	4	1	109	2559	
Octavio Sansegundo Quinto	43	6	1	109	1769	
Zacarias Rico Morcón	42	7	1	108	2136	
Carlos Tercero Madrid	45	2	1	110	1146	
Frutos Campo Dorado	41	2	1	111	2678	
María José Chal Abregado	44	2	1	110	1799	
Amaia Itsasoko Gaztelua	35	4	1	109	1797	
Celso Hórreo Madera	44	7	1	108	2287	
César Pareja Feliz	55	3	2	27.5	0	
José Pepe Perez Quitasol	43	2	2	55.5	743	

13 rows selected.

Efectivamente se muestra que el conductor con el pasaporte = 'ESP>>101010110110' tiene un año activo más

- Cambiamos la antigüedad

Para esta prueba actualizaremos el valor de un nuevo conductor a **drivers** para comprobar si sus años de contrato mostrados con la consulta se ven alterados según lo esperado. La actualización es la siguiente:

```
update drivers set cont_start = '22-NOV-20' where passport = 'ESP>>101010110110';
```

El resultado obtenido:

NOMBRE_COMPLETO	EDAD	ANTIGÜEDAD	AÑOS_ACTIVO	MEDIA_PARADAS	MEDIA_PRESTAMOS	PORCENTAJE_NO_DEVUELTOS
-----	-----	-----	-----	-----	-----	-----
Maria Pensamiento Calma Grata	42	7	1			
110 1788						
Victoria Rojo Blanco	38	2	1			
109 1664						
Fuencisla Maja San Frutos	33	5	1			
108 2599						
Eulalia Puig Castella	39	4	1			
109 2559						
Octavio Sansegundo Quinto	43	6	1			
109 1769						
Zacarías Rico Morcón	42	7	1			
108 2136						
Carlos Tercero Madrid	45	2	1			
110 1146						
Frutos Campo Dorado	41	2	1			
111 2678						
Maria José Chal Abridado	44	2	1			
110 1799						
Amaia Itsasoko Gaztelua	35	4	1			
109 1797						
Celso Hórreo Madera	44	7	1			
108 2287						
NOMBRE_COMPLETO	EDAD	ANTIGÜEDAD	AÑOS_ACTIVO	MEDIA_PARADAS	MEDIA_PRESTAMOS	PORCENTAJE_NO_DEVUELTOS
-----	-----	-----	-----	-----	-----	-----
César Pareja Feliz	55	4	2			
27.5 0						
José Pepe Perez Quitasol	43	2	2			
55.5 743						

13 rows selected.

Vemos que la antigüedad del conductor con el passport = 'ESP>>101010110110' ha variado

- Inserción de ruta

Para esta prueba insertaremos una nueva ruta a un conductor, que suponen más paradas, para comprobar si para dicho conductor el número de paradas medio se ve aumentado:

```
insert into assign_drv values('ESP>>101010110110', '04-APR-23', 'IC-04');
```

El resultado obtenido:

NOMBRE_COMPLETO	EDAD	ANTIGÜEDAD	AÑOS_ACTIVO	MEDIA_PARADAS	MEDIA_PRESTAMOS	PORCENTAJE_NO_DEVUELTOS
-----	-----	-----	-----	-----	-----	-----
Maria Pensamiento Calma Grata	42	7	1			
110 1788						
Victoria Rojo Blanco	38	2	1			
109 1664						
Fuencisla Maja San Frutos	33	5	1			
108 2599						
Eulalia Puig Castella	39	4	1			
109 2559						
Octavio Sansegundo Quinto	43	6	1			
109 1769						
Zacarías Rico Morcón	42	7	1			
108 2136						
Carlos Tercero Madrid	45	2	1			
110 1146						
Frutos Campo Dorado	41	2	1			
111 2678						
Maria José Chal Abridado	44	2	1			
110 1799						
Amaia Itsasoko Gaztelua	35	4	1			
109 1797						
Celso Hórreo Madera	44	7	1			
108 2287						
NOMBRE_COMPLETO	EDAD	ANTIGÜEDAD	AÑOS_ACTIVO	MEDIA_PARADAS	MEDIA_PRESTAMOS	PORCENTAJE_NO_DEVUELTOS
-----	-----	-----	-----	-----	-----	-----
César Pareja Feliz	55	4	2			
27.5 0						
José Pepe Perez Quitasol	43	2	2			
55.5 743						

13 rows selected.

Efectivamente, se muestra un cambio en el número medio de paradas del conductor con el pasaporte: 'ESP>>101010110110'.

- Inserción de préstamos

Para esta prueba insertaremos nuevos préstamos dentro de la ruta del conductor para ver si su media de préstamos por año se ve afectado:

```
insert into loans(signature, user_Id, stopdate, town, province, type, time) values('SN302',  
'0110647937' , trunc(sysdate), 'Navas del Pinar', 'Cadiz', 'L' ,1040);  
insert into loans(signature, user_Id, stopdate, town, province, type, time) values('SN303',  
'0110647937' , trunc(sysdate)+14, 'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'R' ,  
600);  
insert into assign_drv values ('ESP>>101010101010', trunc(sysdate), 'AN-11');  
insert into assign_bus values ('BUS-017', trunc(sysdate), 'AN-11');  
insert into services(town, province, bus, taskdate, passport) values ('Navas del Pinar', 'Cádiz',  
'BUS-017', trunc(sysdate), 'ESP>>101010101010');
```

El resultado obtenido:

No se ha podido realizar la prueba por problemas a la hora de borrar el user actual de la tabla users.

Efectivamente se muestra un cambio en el número medio de préstamos del conductor con el pasaporte: 'ESP>>101010110110'.

- Inserción de préstamos no devueltos

Para esta prueba insertaremos nuevos préstamos no devueltos dentro de la ruta del conductor para ver si su porcentaje de préstamos no devueltos por año se ve afectado:

No se ha podido realizar la prueba por problemas a la hora de insertar.

3. Paquete

En este apartado hemos creado un paquete **foundicu** con sus procedimientos, que se desarrollan más adelante.

```
CREATE OR REPLACE PACKAGE foundicu IS
```

```
    PROCEDURE insertar_prestamo;
```

```
    PROCEDURE insertar_reserva;
```

```
    PROCEDURE registrar_devolucion;
```

```
END foundicu;
```

Procedimiento 1: insertar_prestamo:

Este procedimiento tiene como objetivo realizar las inserciones de los préstamos, comprobando en su ejecución que se cumplan ciertos requisitos para poder realizarlo (límite de préstamos, sanción impuesta a un usuario, etc...).

a) Diseño:

Este procedimiento recibe como entrada una signatura. La primera verificación que se debe hacer es comprobar que el usuario que quiere realizar el préstamo, en este caso el usuario actual, exista. Una vez comprobado eso verificará si el día que se quiere realizar el préstamo se encuentra un bibus en la zona, ya que se presupone que estos solo podrán realizarse si para uno en la zona. Si es así, mirará si existe una reserva para el día actual a nombre de ese usuario, realizando una actualización a préstamo en caso afirmativo.

En caso contrario, mirará si la copia está disponible, si el usuario no ha alcanzado el límite máximo de préstamos y si no se encuentra baneado. Si pasa todas las pruebas, insertará el préstamo con los datos necesarios.

Hacemos una distinción entre los préstamos a personas y préstamos a bibliotecas puesto que su límite es distinto.

En cuanto a las salidas, lo que se producirá será una actualización o un préstamo en caso de que se cumplan todos los requisitos o saltarán las excepciones correspondientes a cada error.

Las excepciones que pueden salir son las siguientes:

- user_no_encontrado: salta cuando no se ha encontrado el usuario actual
- no_pasa_bibus: salta cuando no tienes ninguna instalación cerca donde efectuar el préstamo
- distinto_dia_reserva: salta cuando no hay reservas con fecha de hoy
- libro_ya_prestado: salta cuando el libro ya se encuentra prestado con otro usuario
- usuario_sancionado: salta cuando el usuario actual se encuentra sancionado
- limite_prestamos: salta cuando el préstamo supera el límite permitido
- reservado_por_otro: salta cuando el libro se encuentra reservado por otra persona
- OTHERS: salta cuando ocurre algún otro error

b) Implementación en SQL:

```
CREATE OR REPLACE PROCEDURE insertar_prestamo (input_signatura char) IS
    usuario NUMBER;
    dia_reserva NUMBER;
    dia_reserva2 DATE;
    otro_user CHAR;
    reservas NUMBER;
    prestamos NUMBER;
    sancionado DATE;
    ciudad VARCHAR2(50);
    provincia VARCHAR2(22);
    hay_bibus NUMBER;
    tipo_user CHAR;
    num_habitantes NUMBER;
    hora NUMBER;
    user_no_encontrado EXCEPTION;
    no_pasa_bibus EXCEPTION;
    distinto_dia_reserva EXCEPTION;
    libro_ya_prestado EXCEPTION;
    usuario_sancionado EXCEPTION;
    limite_prestamos EXCEPTION;
    reservado_por_otro EXCEPTION;
BEGIN
    select count('x') into usuario from users where user_id = USER;
    if usuario = 0
    then raise user_no_encontrado;
    end if;

    select count('x') into dia_reserva from loans where signature = input_signatura and
    user_id = USER and stopdate = trunc(sysdate) and type = 'R';
    select town, province into ciudad, provincia from users where user_id = USER;
    select count('x') into hay_bibus from services where town = ciudad and province =
    provincia and taskdate = trunc(sysdate);
    if hay_bibus = 0 then
    raise no_pasa_bibus;
    end if;

    if dia_reserva > 0 then
        update loans set type = 'L', return = trunc(sysdate) + 14 where
        signature = input_signatura and user_id = USER and stopdate = trunc(sysdate);
```

```
else
    select stopdate into dia_reserva2 from loans where signature = input_signatura and
user_id = USER and type = 'R';
    if dia_reserva2 is not NULL then
        raise distinto_dia_reserva;
    end if;

    select user_id into otro_user from loans where signature = input_signatura and type =
'L' and return >= trunc(sysdate);
    if otro_user is not null then
        raise libro_ya_prestado;
    end if;

    select count('x') into reservas from loans where signature = input_signatura and type =
'R' and stopdate < trunc(sysdate)+14;
    if reservas > 0 then
        raise reservado_por_otro;
    end if;
    select type into tipo_user from users where user_id = USER;
    if tipo_user = 'P' then
        select count('x') into prestamos from loans where user_id=USER and type = 'L' and
return >= trunc(sysdate);
        if prestamos < 2 then
            select ban_up2 into sancionado from users where user_id=USER;
            if sancionado is NULL or sancionado >= trunc(sysdate) then
                raise usuario_sancionado;
            end if;
            select stoptime into hora from stops where town=ciudad and province=provincia;
            insert into loans values(input_signatura, USER, trunc(sysdate), ciudad, provincia, 'L',
hora, trunc(sysdate) + 14, NULL);
        else
            raise limite_prestamos;
        end if;

    else
        select count('x') into prestamos from loans where user_id=USER and type = 'L' and
return >= trunc(sysdate);
        select population into num_habitantes from municipalities where town= ciudad and
province = provincia;
        if prestamos < floor((num_habitantes*2)/10) then
```

```
select stoptime into hora from stops where town = ciudad and province = provincia;
insert into loans values(input_signatura, USER, trunc(sysdate), ciudad, provincia, 'L',
hora, trunc(sysdate) + 14, NULL);
        else
            raise limite_prestamos;
        end if;
    end if;
end if;
EXCEPTION
when user_no_encontrado then RAISE_APPLICATION_ERROR (-20001, 'No se ha
encontrado el usuario actual');
when no_pasa_bibus then RAISE_APPLICATION_ERROR (-20002, 'No tienes
ninguna instalación cerca donde efectuar el préstamo');
when distinto_dia_reserva then RAISE_APPLICATION_ERROR (-20003, 'No hay
reservas con fecha de hoy');
when libro_ya_prestado then RAISE_APPLICATION_ERROR (-20004, 'El libro ya se
encuentra prestado con otro usuario');
when usuario_sancionado then RAISE_APPLICATION_ERROR (-20005, 'El usuario
actual se encuentra sancionado');
when limite_prestamos then RAISE_APPLICATION_ERROR (-20006, 'Este prestamo
supera el limite permitido');
when reservado_por_otro then RAISE_APPLICATION_ERROR (-20007, 'El libro se
encuentra reservado por otra persona');
END;
```

c) Batería de pruebas:

Para la batería de pruebas probaremos a ejecutar el procedimiento introduciendo casos válidos y todos aquellos casos por los que debería saltar una excepción: No se han podido realizar las pruebas porque no ha sido posible eliminar el user actual de la tabla, ya que el programa se quedaba colgado. Se dejan indicados

- Caso válido: actualización de un préstamo con una reserva previa:

```
insert into assign_drv values('ESP>>101010110110', '04-APR-25', 'IC-04');
insert into users(user_id, id_card, name, surname1, birthdate, town, province, address,
phone, type) values('fsdb134' , 'ESP>>340110647301', 'Peppa', 'Pig', '30-JUL2005',
'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'Calle de los Cerditos',
666666668, 'P');
insert into loans(signature, user_Id, stopdate, town, province, type, time)
values('SN301', 'fsdb134' , sysdate, 'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'R' ,
600);
execute insertar_prestamo('UJ596');
```



```
insert into assign_drv values ('ESP>>101010101010', trunc(sysdate), 'AN-11');
insert into assign_bus values ('BUS-017', trunc(sysdate), 'AN-11');
insert into services(town, province, bus, taskdate, passport) values ('Navas del Pinar', 'Cádiz',
'BUS-017', trunc(sysdate), 'ESP>>101010101010');
insert into loans values ('UJ596', USER, trunc(sysdate), 'Navas del Pinar', 'Cádiz', 'R', 1040,
NULL, NULL);
select * from assign_drv where passport='ESP>>101010101010' and taskdate=trunc(sysdate);
```

- **Caso válido: inserción de un préstamo para una persona sin reserva previa:**
insert into assign_drv values('ESP>>101010110110', '04-APR-25', 'IC-04');
insert into users(user_id, id_card, name, surname1, birthdate, town, province, address, phone, type) values('fsdb134', 'ESP>>340110647301', 'Peppa', 'Pig', '30/07/2005', 'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'Calle de los Cerditos', 666666668, 'P');
delete from loans where signature = 'SN301';
- **Caso válido: inserción de un préstamo para una biblioteca sin reserva previa:**
insert into assign_drv values('ESP>>101010110110', '04-APR-25', 'IC-04');
delete from users where user_id = USER;
insert into users(user_id, id_card, name, surname1, birthdate, town, province, address, phone, type) values('fsdb134', 'ESP>>340110647301', 'Peppa', 'Pig', '30/07/2005', 'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'Calle de los Cerditos', 666666668, 'L');
delete from loans where signature = 'SN301';
- **Caso no válido: USER no encontrado:**
insert into loans(signature, user_Id, stopdate, town, province, type, time) values('SN301', 'fsdb134', sysdate, 'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'R', 600);
- **Caso no válido: no hay instalaciones en el día de hoy**
delete from loans where signature = 'SN301';
insert into loans(signature, user_Id, stopdate, town, province, type, time) values('SN301', 'fsdb134', sysdate, 'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'R', 600);
delete from assign_drv where taskdate = '04-APR-25';
- **Caso no válido: no hay reservas con fecha actual:**
delete from loans where signature = 'SN301';
insert into loans(signature, user_Id, stopdate, town, province, type, time) values('SN301', 'fsdb134', sysdate+14, 'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'R', 600);
- **Caso no válido: El libro está prestado en este momento:**

```
delete from loans where signature = 'SN302';  
insert into loans(signature, user_Id, stopdate, town, province, type, time) values('SN302',  
'0110647937', sysdate, 'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'L', 600);
```

- **Caso no válido: usuario sancionado**

```
insert into assign_drv values('ESP>>101010110110', '04-APR-25', 'IC-04');  
delete from users where user_id = 'fsdb134';  
insert into users(user_id, id_card, name, surname1, birthdate, town, province, address, phone,  
type, ban_up2) values('fsdb134', 'ESP>>340110647301', 'Peppa', 'Pig', '30/07/2005', 'Valle de  
la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'Calle de los Cerditos', 666666668, 'P',  
'05-MAY-25');
```

```
delete from loans where signature = 'SN301';
```

- **Caso no válido: usuario persona límite préstamos**

```
delete from loans where signature = 'SN302';  
delete from loans where signature = 'SN301';  
insert into loans(signature, user_Id, stopdate, town, province, type, time) values('SN302',  
'fsdb134', '21-MAR-25', 'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'L', 600);  
insert into loans(signature, user_Id, stopdate, town, province, type, time) values('SN301',  
'fsdb134', '21-MAR-25', 'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'L', 600);
```

- **Caso no válido: usuario biblioteca límite préstamos**

```
delete from loans where signature = 'SN302';  
delete from loans where signature = 'SN301';
```

```
insert into loans(signature, user_Id, stopdate, town, province, type, time) values('SN302',  
'fsdb134', '21-MAR-25', 'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'L', 600);  
insert into loans(signature, user_Id, stopdate, town, province, type, time) values('SN301',  
'fsdb134', '21-MAR-25', 'Valle de la Costa (Tenerife)', 'Santa Cruz de Tenerife', 'L', 600);  
delete from users where user_id = 'fsdb134';  
insert into users(user_id, id_card, name, surname1, birthdate, town, province, address, phone,  
type) values('fsdb134', 'ESP>>340110647301', 'Peppa', 'Pig', '30/07/2005', 'Valle de la Costa  
(Tenerife)', 'Santa Cruz de Tenerife', 'Calle de los Cerditos', 666666668, 'L');
```

Procedimiento 2: insertar reserva:

Este procedimiento tiene como objetivo realizar las inserciones de las reservas, comprobando en su ejecución que se cumplan ciertos requisitos para poder realizarlo (límite de préstamos, sanción impuesta a un usuario, etc...).

a) Diseño:

Este procedimiento recibe como entrada un isbn y una fecha. Lo primero que se comprueba es si el usuario que quiere realizar el préstamo, en este caso el usuario actual, existe. Una vez comprobado eso verificará si el usuario ha alcanzado o no el límite de préstamos. Si no lo ha alcanzado, verifica si está sancionado.

Si estos requisitos se cumplen, entonces mira si existe una copia disponible para la fecha solicitada y tiene dos semanas de disponibilidad para poder realizar el préstamo sin interferir en las reservas de otros usuarios. Si se encuentra alguna copia disponible hará la reserva, en caso contrario o de algún error anterior, informará de los problemas. Además también se comprueba si para el día que se hace la reserva hay servicio en ese lugar, puesto que si no, la reserva no se podría coger.

Al igual que con el procedimiento anterior, hacemos una distinción entre los préstamos a personas y préstamos a bibliotecas puesto que su límite es distinto.

En cuanto a las salidas, se producirá una inserción de una reserva en caso de que se cumplan todos los requisitos o mostrará por pantalla el fallo que ha saltado.

Las fallos que pueden salir son los siguientes:

- user_no_encontrado: se imprime cuando no existe el user actual
- limite_prestamos: se imprime cuando se ha alcanzado el límite de préstamos
- usuario_sancionado: se imprime cuando el usuario que está solicitando la reserva está sancionado
- no_copias_disponibles: se imprime cuando no hay copias disponibles para esa fecha
- no_hay_servicio: se imprime cuando no hay servicio para la fecha solicitada
- OTHERS: se imprime cuando salta algún otro error

b) Implementación en SQL:

```
CREATE OR REPLACE PROCEDURE insertar_reserva (input_isbn VARCHAR2,  
fecha DATE) IS
```

```
    usuario NUMBER;  
    num_prestamos NUMBER;  
    num_poblacion NUMBER;  
    ciudad VARCHAR2 (50);  
    provincia VARCHAR2 (22);  
    sancion DATE;  
    hay_servicio NUMBER;  
    hora NUMBER;  
    signatura_disponible CHAR(5);  
    user_no_encontrado EXCEPTION;
```

```
limite_prestamos EXCEPTION;  
usuario_sancionado EXCEPTION;  
no_copias_disponibles EXCEPTION;  
no_hay_servicio EXCEPTION;  
  
BEGIN  
    SELECT count('x') INTO usuario FROM users WHERE user_id=USER AND  
type='p';  
    IF usuario = 0 THEN  
        SELECT count('x') INTO usuario FROM users WHERE user_id=USER AND type='L';  
        IF usuario = 0  
            THEN RAISE user_no_encontrado;  
        ELSE  
            SELECT count('x') INTO num_prestamos FROM loans WHERE user_id=USER AND  
devolucion IS NULL;  
            SELECT town, province INTO ciudad, provincia FROM users WHERE  
user_id=USER;  
            SELECT population INTO num_poblacion FROM municipalities WHERE  
town=ciudad AND province=provincia;  
  
            IF floor((2*num_poblacion/10)) = num_prestamos  
                THEN RAISE limite_prestamos;  
            END IF;  
        END IF;  
    ELSE  
        SELECT count('x') INTO num_prestamos FROM loans WHERE user_id=USER AND  
devolucion IS NULL;  
        IF num_prestamos >= 2  
            THEN RAISE limite_prestamos;  
        END IF;  
    END IF;  
  
    SELECT ban_up2 INTO sancion FROM users WHERE user_id=USER;  
    IF sancion IS NOT NULL  
        THEN RAISE usuario_sancionado;  
    END IF;  
    with  
    A as (select distinct signature from Copies where isbn=input_isbn),  
    B as (select distinct signature from Loans where (type='L' and devolucion is null) or  
(type='R' and stopdate<fecha+14)),
```

```
C as (select distinct A.signature from A LEFT JOIN B ON A.signature=B.signature
WHERE B.signature IS NULL)
select signature into signatura_disponible from C where rownum=1;
```

```
IF signatura_disponible IS NULL
    THEN RAISE no_copias_disponibles;
END IF;
```

```
SELECT town, province INTO ciudad, provincia FROM users WHERE
user_id=USER;
SELECT count('x') INTO hay_servicio FROM services where town=ciudad AND
province=provincia AND taskdate=fecha;
IF hay_servicio = 0
    THEN RAISE no_hay_servicio;
END IF;
```

```
SELECT stoptime INTO hora FROM stops WHERE town=ciudad AND
province=provincia;
```

```
INSERT INTO loans VALUES (signatura_disponible, USER, fecha, ciudad, provincia,
'R', hora, NULL, NULL);
EXCEPTION
    WHEN user_no_encontrado THEN dbms_output.put_line('no user');
    WHEN limite_prestamos THEN dbms_output.put_line('limite prestamos');
    WHEN usuario_sancionado THEN dbms_output.put_line('usuario sancionado');
    WHEN no_copias_disponibles THEN dbms_output.put_line('no copias
disponibles');
    WHEN no_hay_servicio THEN dbms_output.put_line('no hay servicio para ese
dia');
    WHEN OTHERS THEN dbms_output.put_line('se ha producido un error');
END;
/
```

c) Batería de pruebas:

Para la batería de pruebas probaremos a ejecutar el procedimiento introduciendo casos válidos y todos aquellos casos por los que debería saltar una excepción: No se han podido realizar las pruebas porque no ha sido posible eliminar el user actual de la tabla, ya que el programa se quedaba colgado. Se dejan indicados

- Caso válido: usuario persona hace reserva
- Caso válido: usuario biblioteca hace reserva
- Caso no válido: user no existe
- Caso no válido: no hay servicio para esa fecha
- Caso no válido: pasa limite de préstamos
- Caso no válido: usuario sancionado
- Caso no válido: ninguna copia disponible

Procedimiento 3: registrar_devolución:

Este procedimiento tiene como objetivo realizar los registros de aquellos libros que se han devuelto.

a) Diseño:

Este procedimiento recibe como entrada una signatura. Primero verifica si el USER actual existe, y si lo hace comprueba si tiene ese libro prestado. Si lo tiene actualiza los datos del préstamo para poner en el atributo devolución la fecha en la que se ha producido la devolución.

Como el enunciado no lo especifica, no se ha tratado las sanciones para el usuario, pero en caso de tener que hacerlo, deberíamos comprobar si la fecha en la que se ha devuelto es superior a la que está guardada en el atributo return, en caso de serlo, se haría la resta de sysdate menos esa fecha para calcular las semanas de sanción. Estas semanas se sumarían a sysdate y se quedaría la fecha resultante guardada en el atributo ban_up2 de users para saber si el usuario ha sido baneado.

En cuanto a las salidas, se producirá una actualización del préstamo para indicar que se ha actualizado la fecha y lanzará excepciones si salta algún fallo durante su ejecución.

Las excepciones que pueden salir son las siguientes:

- user_no_encontrado: salta cuando no existe el user actual
- libro_no_prestado: salta cuando el libro no se encuentra prestado para ese usuario
- prestado_mas_de_una: salta en caso de que se hubiera registra varios préstamos a nombre de ese usuario para la misma copia
- OTHERS: se imprime cuando salta algún otro error

b) Implementación en SQL:

```
CREATE OR REPLACE PROCEDURE registrar_devolucion(input_signatura CHAR)
IS
```

```
    usuario NUMBER;
    libro_prestado NUMBER;
    user_no_encontrado EXCEPTION;
    libro_no_prestado EXCEPTION;
    prestado_mas_de_una EXCEPTION;
```

```
BEGIN
    SELECT count('x') INTO usuario FROM users WHERE user_id=USER;
    IF usuario = 0
        THEN RAISE user_no_encontrado;
    END IF;

    SELECT count('x') INTO libro_prestado FROM loans WHERE
signature=input_signatura AND user_id=USER AND devolucion IS NULL;
    IF libro_prestado = 0
        THEN RAISE libro_no_prestado;
    END IF;
    IF libro_prestado > 1
        THEN RAISE prestado_mas_de_una;
    END IF;

    UPDATE loans SET devolucion =sysdate WHERE signature=input_signatura AND
user_id=USER AND devolucion IS NULL;

EXCEPTION
    WHEN user_no_encontrado THEN RAISE_APPLICATION_ERROR (-20008, 'No se
ha encontrado el usuario actual');
    WHEN libro_no_prestado THEN RAISE_APPLICATION_ERROR (-20009, 'No tienes
el libro prestado');
    WHEN prestado_mas_de_una THEN RAISE_APPLICATION_ERROR (-20010,
'Tienes el libro prestado más de una vez al mismo tiempo');
    WHEN OTHERS THEN RAISE_APPLICATION_ERROR (-20011, 'Se ha producido
otro error');
END;
```

c) Batería de pruebas:

Para la batería de pruebas probaremos a ejecutar el procedimiento introduciendo casos válidos y todos aquellos casos por los que debería saltar una excepción: No se han podido realizar las pruebas porque no ha sido posible eliminar el user actual de la tabla, ya que el programa se quedaba colgado. Se dejan indicados:

Titulación: GRADO EN INGENIERÍA INFORMÁTICA
Año Académico: 2024/25 – 2º cuatrimestre
Asignatura: Ficheros y bases de datos (2º curso)

Memoria de Prácticas 2: Consultas y Extensiones Procedimentales



- Caso válido: usuario devuelve libro
- Caso no válido: user no existe
- Caso no válido: user no tiene ese libro prestado
- Caso no válido: ese libro aparece prestado varias veces

4. Diseño externo

Describe las vistas y realiza su diseño, implementación y pruebas (de modo análogo a como se hicieran las consultas en el apartado 2, pero desarrollando su completitud funcional si fuera necesario). Incluye una subsección por cada vista que desarrolles, exponiendo:

Vista 1: my_data:

Esta vista permite al usuario actual visualizar toda su información personal. Es una vista de solo lectura, por lo que no se podrán hacer actualizaciones sobre ella.

a) Diseño en álgebra relacional:

No consideramos que sea necesario ya que es una consulta simple

b) Implementación en SQL:

La vista consta de una consulta, que será la que se visualice cuando se consulte **my_data**, y cuenta con la etiqueta **WITH READ ONLY**, que hace que sea no operable.

```
CREATE OR REPLACE VIEW my_data AS
select distinct * from users where users.user_id=USER
WITH READ ONLY;
```

Esto es lo que se muestra por pantalla:

USER_ID	ID_CARD	NAME	SURNAME1	SURNAME2
FSDB134	ESP>>340110647938	Marta	Vindel	Navas

BIRTHDATE	TOWN	PROVINCE	ADDRESS
30-JUL-05	Navas del Pinar	Cádiz	su casa

EMAIL	PHONE	T	BAN_UP2
sucorreo@gmail.com	111111111	p	

c) Batería de pruebas:

Para comprobar que la consulta funciona correctamente y muestra los resultados esperados por pantalla, realizamos las siguientes pruebas:

- Insertar USER actual:

Puesto que partimos de una base de datos en la que no está registrado nuestro usuario, lo primero que hacemos para comprobar esta vista es insertarlo en **users**. De esta manera comprobamos que se muestra la información personal del usuario.

La inserción es:

```
insert into users values(USER, 'ESP>>340110647938', 'Felicia', 'Pareja', 'Triste',  
'30-JUL-2005', 'Navas del Pinar', 'Cádiz', 'Mi casa', 'micorreo@gmail.com',  
123456789, 'P', NULL);
```

El resultado obtenido es:

No se ha podido realizar la prueba por problemas a la hora de borrar el user actual de la tabla users, daba fallo la clave primaria.

- Actualizar información del USER actual

A continuación, modificamos algún dato de nuestro usuario y comprobamos que quede reflejado el cambio en la vista. La modificación es:

```
update users set email='nuevocorreo@gmail.com' where user_id=USER;
```

El resultado obtenido es:

EMAIL	PHONE T	BAN_UP2
nuevocorreo@gmail.com	111111111	p

- Eliminar USER actual

Por último, eliminamos la fila con nuestro user de **users** y comprobamos que ya no se vea en la vista. El borrado es:

```
delete from users where user_id=USER;
```

El resultado obtenido es:

No se ha podido realizar la prueba por problemas a la hora de borrar el user actual de la tabla users, se quedaba pillado

Puesto que my_data es de solo lectura, también haremos casos para comprobar que no permite ser actualizada. Si la vista está bien hecha, todas estas pruebas deberán devolver un error:

- Insertar en la vista:

Sin ningún dato en la vista, tratamos de insertar una fila con datos de nuestro USER. La inserción es:

```
insert into my_data values(USER, 'ESP>>340110647938', 'Felicia', 'Pareja', 'Triste',  
'30/07/2005', 'Navas del Pinar', 'Cádiz', 'Mi casa', 'micorreo@gmail.com', 123456789,  
'P', NULL);
```

Efectivamente, se produce un error al ejecutar esta instrucción:

```
SQL> insert into my_data values(USER, 'ESP>>340110647938', 'Felicia', 'Pareja', 'Triste', '30/07/2005', 'Navas del Pinar', 'Cádiz', 'Mi casa', 'micorreo@gmail.com', 123456789, 'P', NULL);  
insert into my_data values(USER, 'ESP>>340110647938', 'Felicia', 'Pareja', 'Triste', '30/07/2005', 'Navas del Pinar', 'Cádiz', 'Mi casa', 'micorreo@gmail.com', 123456789, 'P', NULL)  
ERROR at line 1:  
ORA-42309: cannot perform a DML operation on a read-only view
```

- Modificar la vista:

Con nuestro usuario ya registrado, tratamos de modificar el valor de alguno de sus campos. La modificación es:

```
update my_data set phone=987654321 where user_id=USER;
```

De nuevo, nos sale el error esperado:

```
SQL> update my_data set phone=987654321 where user_id=USER;
update my_data set phone=987654321 where user_id=USER
      *
ERROR at line 1:
ORA-42399: cannot perform a DML operation on a read-only view
```

- Eliminar de la vista:

Tratamos de eliminar de la vista nuestro usuario ya creado. El borrado es:

```
delete from my_data where user_id=USER;
```

Obtenemos un error:

```
SQL> delete from my_data where user_id=USER;
delete from my_data where user_id=USER
      *
ERROR at line 1:
ORA-42399: cannot perform a DML operation on a read-only view
```

Vista 2: my_loans:

Esta vista permite al usuario actual visualizar todos sus préstamos presentes y pasados. También muestra los comentarios hechos por el usuario para cada uno de esos préstamos. **My_loans** sí será operable, y permitirá la modificación del texto de las publicaciones, actualizando consigo la fecha del post. Si la publicación no existiese, inicializaría los likes y dislikes a 0; de no ser así, los deja como estaban.

a) Diseño en álgebra relacional:

La vista en sí consiste de una combinación externa por la izquierda de préstamos y posts, de modo que se visualicen todos los préstamos, incluso aquellos que no tengan publicaciones, cuyos atributos aparecerán como valores nulos.

b) Implementación en SQL:

Esta vista consta de una selección, ya explicada en el apartado anterior, y la etiqueta **WITH CHECK OPTION**, que impide que se realicen actualizaciones en las que algún elemento modificado sería excluido de la vista.

```
CREATE OR REPLACE VIEW my_loans AS
    select * from (select * from loans where user_id=USER and stopdate < sysdate)
left join posts
    using(signature, user_id, stopdate)
WITH CHECK OPTION;
```

Además, hemos creado dos triggers. El primero impide cualquier inserción o borrado en la vista, tal y como pide el enunciado. Para esto utilizamos un disparador que en vez de realizar estas operaciones, lanza un error personalizado.

```
CREATE OR REPLACE TRIGGER insert_delete_my_loans
INSTEAD OF INSERT OR DELETE ON my_loans
FOR EACH ROW
DECLARE
    post_error EXCEPTION;
BEGIN
    RAISE post_error;
EXCEPTION
    WHEN post_error THEN RAISE_APPLICATION_ERROR (-20002, 'No se puede
insertar ni eliminar filas en esta vista');
END insert_delete_my_loans;
```

```
CREATE OR REPLACE TRIGGER update_post_my_loans
INSTEAD OF UPDATE on my_loans
```

```

for each row
DECLARE
  no_update EXCEPTION;
BEGIN
  IF :OLD.text=:NEW.text OR :OLD.signature!=:NEW.signature OR
  :OLD.user_id!=:NEW.user_id OR :OLD.stopdate!=:NEW.stopdate
  OR :OLD.town!=:NEW.town OR :OLD.province!=:NEW.province OR
  :OLD.type!=:NEW.type OR :OLD.time!=:NEW.time
  OR :OLD.return!=:NEW.return OR :OLD.post_date!=:NEW.post_date OR
  :OLD.likes!=:NEW.likes OR :OLD.dislikes!=:NEW.dislikes
  THEN
    raise no_update;
  ELSE
    IF :OLD.text is NULL THEN
      insert into posts values(:OLD.signature, :OLD.user_id, :OLD.stopdate,
      sysdate, :NEW.text, 0, 0);
    ELSE
      update posts set text=:NEW.text where signature=:OLD.signature and
      user_id=:OLD.user_id and stopdate=:OLD.stopdate;
    END IF;
  END IF;
EXCEPTION
  WHEN no_update THEN RAISE_APPLICATION_ERROR (-20003, 'Solo se
  puede modificar el texto del post');
END update_post_my_loans;
  
```

Puesto que no se puede hacer una modificación en una vista como `my_loans`, que combina dos tablas, hemos creado un trigger que se ejecuta como sustituto de la modificación (INSTEAD OF). Comenzamos comprobando que el único valor que se intenta cambiar es el de `text` y mira si el `posts` existe. Si no existía, inserta uno nuevo en **posts** con los valores iniciales mencionados anteriormente. Por otro lado, si la publicación ya está en la base de datos, entonces se modifican únicamente los valores del texto y la fecha.

Lo que se muestra por pantalla:

SIGNA	USER_ID	STOPDATE	TOWN	PROVINCE	T	TIME	RETURN	DEVOLUCIO	POST_DATE	TEXT	LIKES	DISLIKES
NC628	FSDB134	19-NOV-24	Serranía del Vertedero	Girona	R	720	03-DEC-25					
1B311	FSDB134	16-NOV-24	Sotosombras	Teruel	R	705	30-NOV-24					
U7596	FSDB134	04-APR-25	Navas del Pinar	Cádiz	R	1040						

c) Batería de pruebas:

Para comprobar que la consulta funciona correctamente y muestra los resultados esperados por pantalla, realizamos las siguientes pruebas:

- Insertar un préstamo pasado:

Comenzamos mirando cuántos valores devuelve la vista. A continuación, insertamos un préstamo pasado con nuestro usuario actual en **loans** y observamos que la vista devuelve un valor más.

```
insert into loans values('SN30', USER, '22-NOV-24', 'Navas del Pinar', 'Cádiz', 'L', 1040, NULL, NULL);
```

No se ha podido realizar la prueba por problemas a la hora de borrar el user actual de la tabla users.

- Insertar un préstamo futuro (reserva):

Igualmente, miramos cuántos valores devuelve la vista, y después insertamos en **loans** un préstamo futuro con nuestro usuario actual y observamos que la vista devuelve el mismo número de valores.

El número de filas en la vista no cambia:

No se ha podido realizar la prueba por problemas a la hora de borrar el user actual de la tabla users.

- Eliminar un préstamo:

Comenzamos mirando cuántos valores devuelve la vista. A continuación, eliminamos de **loans** uno de los préstamos de **my_loans** y observamos que la vista devuelve un valor menos.

Para comprobar que los disparadores asociados a **my_loans** funcionan correctamente realizamos las pruebas:

- Insertar en la vista:

Al tratar de insertar una fila en la vista se activará el trigger **insert_delete_my_loans**, que lanzará un error.

- Eliminar de la vista:

Lo mismo sucede cuando tratamos de borrar una fila. Se activará **insert_delete_my_loans** y se volverá a producir el error que buscamos.

- Modificar texto de un post existente en la vista:
Si modificamos el texto de un post ya existente de la vista, observaremos que lo ha modificado, junto con su fecha. El resto de valores permanecerán iguales.
- Modificar texto de un post no existente en la vista:
Si modificamos el texto de un post con valor nulo, deberíamos observar una modificación en la vista, con el nuevo texto, la fecha de hoy y likes y dislikes a cero. También se tendría que ver el nuevo post en **posts**.
- Modificar otros valores en la vista:
Si tratamos de modificar un valor que no sea el texto, se producirá un error y no se realizará el cambio.

Vista 3: my_reservations:

Esta vista permite al usuario actual visualizar todas sus reservas. **My_reservations** es operable, y permite la inserción, el borrado y la modificación de las reservas, siempre y cuando se cumplan las condiciones especificadas en el enunciado.

- a) Diseño en álgebra relacional:
La consulta de esta vista simplemente recoge todos los valores de las reservas realizadas por el usuario.
- b) Implementación en SQL:
Al igual que la vista **my_loans**, está es operable, por lo que utilizamos **WITH CHECK OPTION** para garantizar que la vista es pura.

```
CREATE OR REPLACE VIEW my_reservations AS
    select * from loans where user_id=USER and type='R'
WITH CHECK OPTION;
```

Para permitir las actualizaciones hemos creado distintos triggers. En ellos hemos utilizado el segundo procedimiento del paquete (**insertar_reserva**), que se encarga de revisar que se cumplan los requisitos necesarios para la operación y, de ser así, la ejecuta.

Para las inserciones hemos utilizado el disparador **insert_my_reservations**, en el que obtenemos el ISBN de la copia y llamamos al procedimiento para que realice la inserción.

Puesto que **my_reservations** es operable y solo consulta una tabla base, se pueden hacer los borrados inmediatamente, sin la necesidad de un trigger.

El disparador para la modificación lo hemos considerado aparte de la inserción. Esto se debe a que en la actualización seguimos un camino algo diferente: guardamos el valor de la reserva en una variable auxiliar, después la eliminamos de **loans** y por último, tratamos de insertar una reserva nueva con los datos nuevos. Si **insertar_reserva** falla, quiere decir que no se ha podido insertar ninguna reserva por falta de disponibilidad, así que reinsertamos el valor original de nuestra reserva, indicando que no se ha podido realizar la modificación.

```
CREATE OR REPLACE TRIGGER insert_my_reservations
INSTEAD OF INSERT ON my_reservations
FOR EACH ROW
DECLARE
    aux_isbn VARCHAR(20);
BEGIN
    select isbn into aux_isbn from copies where signature=:NEW.signature;
    insertar_reserva(aux_isbn, sysdate);
END insert_my_reservations;
```

```
CREATE OR REPLACE TRIGGER update_my_reservations
INSTEAD OF UPDATE ON my_reservations
FOR EACH ROW
DECLARE
    aux_isbn VARCHAR(20);
    copia_original loans%rowtype;
    no_update EXCEPTION;
    num_inicial NUMBER;
    num_final NUMBER;
BEGIN
    select * into copia_original from Loans where signature=:OLD.signature and
    user_id=:OLD.user_id and stopdate=:OLD.stopdate;
    select isbn into aux_isbn from copies where signature=:OLD.signature;
    select count('x') into num_inicial from my_reservations;
    delete from loans where signature=:OLD.signature and user_id=:OLD.user_id and
    stopdate=:OLD.stopdate;
    insertar_reserva(aux_isbn, :NEW.stopdate);
    select count('x') into num_final from my_reservations;
```

```
if num_final!=num_inicial THEN
    insert into loans values(copia_original.signature, copia_original.user_id,
copia_original.stopdate, copia_original.town,
    copia_original.province, copia_original.type, copia_original.time,
copia_original.return, copia_original.devolucion);
    raise no_update;
end if;
EXCEPTION
    WHEN no_update THEN dbms_output.put_line('no se ha podido modificar la
reserva');
END update_my_reservations;
```

c) Batería de pruebas:

Para comprobar que la consulta funciona correctamente y muestra los resultados esperados por pantalla, realizamos las siguientes pruebas:

- Insertar una reserva:
Comprobamos que, al insertar una reserva en **loans** perteneciente al usuario actual, el número de filas que se muestran en la vista aumenta.
- Insertar una reserva:
Si eliminamos de **loans** una reserva del usuario, podremos observar que ya no aparece en la vista.

Para comprobar que los disparadores se ejecutan correctamente realizamos las siguientes pruebas:

- Insertar reserva disponible:
Tomamos una copia con un ISBN que tenga alguna copia disponible para reservar. Al insertarla en **my_reservations** se empezará a ejecutar el **insert_my_reservations**, con lo que al finalizar debería aparecer una reserva nueva en **my_reservations** de una copia con el mismo ISBN que la original.
- Insertar reserva no disponible:
Tomamos una copia con un ISBN sin ninguna copia disponible. Igual que en la anterior, se ejecutará el disparador, pero **insertar_reserva** lanzará un error, puesto que no se ha podido insertar.
- Eliminar reserva:
Cuando eliminamos una reserva que está en **my_reservations** se debería borrar de la vista y de **loans** sin ningún tipo de error.

- Modificar reserva con disponibilidad:
Si modificamos la fecha de una reserva de my_reservations cuyo ISBN sí tenga alguna copia disponible, podremos observar que se han producido cambios en la fecha y, posiblemente, la signatura.
- Modificar reserva sin disponibilidad:
No obstante, si tratamos de modificar una reserva a una fecha en la que no tiene disponibilidad, no se producirá ningún cambio en la reserva. Los valores permanecerán iguales, pero se lanzará un mensaje de error.

5. Disparadores

Para cada disparador resuelto se debe incluir una subsección que contenga:

- Descripción del diseño: Tabla a la que está asociado, Evento o eventos en los que se dispara, Temporalidad (antes, después o en vez de), Granularidad (por fila o sentencia), Condición (si la tiene) y Acción (descripción en lenguaje natural).
- Código (en SQL)
- Pruebas

Disparador 1: CHK_clause_date:

a) Diseño:

El primer disparador está asociado a la tabla **posts** y en lo que consiste es en evitar que los usuarios que sean de tipo biblioteca puedan escribir y publicar un post asociado a un libro. Este disparador salta antes de insertar o actualizar el usuario de la tabla post y se ejecuta por cada fila de la tabla.

El disparador actúa de la siguiente manera: se va a insertar un nuevo post o se va a actualizar, el trigger salta, y entonces verifica el tipo del usuario que está entrando. Si el tipo es L significa que el post lo quiere hacer una biblioteca, por lo que lanza una excepción e impide que se realice la operación.

b) Código en SQL:

```
CREATE OR REPLACE TRIGGER evitar_post
Before insert or update of user_ID on posts
for each row
DECLARE usuario CHAR;
        tipo_usuario CHAR;
        bad_post EXCEPTION;

BEGIN
        select type into tipo_usuario from users where user_ID=:NEW.user_ID;
        IF tipo_usuario = 'L'
            THEN RAISE bad_post;
        END IF;
EXCEPTION
WHEN bad_post then RAISE_APPLICATION_ERROR (-20001, 'No se puede hacer
un post siendo una biblioteca');
END evitar_post;
```

c) Batería de pruebas:

- Insertar un post con el usuario biblioteca
- Actualizar un post con el usuario biblioteca
- Insertar un post con el usuario persona
- Actualizar un post con el usuario persona

Disparador 2: evitar_post:

a) Diseño:

El segundo disparador está asociado a la tabla **copies** y en lo que consiste es en establecer la fecha y hora actuales de manera automática cuando se da de baja una copia porque su estado se establece como deteriorado.

Este disparador salta antes de insertar o actualizar la condición del libro en la tabla **copies** y se ejecuta por cada fila de la tabla. Se decidió que también podría saltar cuando se está insertando, porque, aunque no tendría mucho sentido que se registrará un libro que se va a dar de baja, consideramos que puede darse el caso de que sea un libro que se extravió cuando se hizo la base de datos y aunque se va a dar de baja se quiere tener registrado en la base como que sí que estuvo entre las copias disponibles.

El disparador actúa de la siguiente manera: se va a insertar una nueva copia o se va a actualizar, el trigger salta, y entonces verifica el tipo del usuario que está entrando. Si el tipo es L significa que el post lo quiere hacer una biblioteca, por lo que lanza una excepción e impide que se realice la operación.

b) Código en SQL:

```
create or replace TRIGGER baja_copia
BEFORE INSERT OR UPDATE OF condition ON Copies
FOR EACH ROW
BEGIN
    IF :NEW.condition = 'D' AND :OLD.deregistered is null
        THEN :new.deregistered := sysdate;
    END IF;
END baja_copia;
```

c) Batería de pruebas:

- Cambiar condición a deteriorado
- Cambiar condición a otro estado
- Insertar un libro deteriorado
- Insertar un libro bueno

Disparador 3: historicos:

a) Diseño:

El tercer y último disparador está asociado a dos tablas, la tabla **users** y la tabla **loans**. Sin embargo, antes de poder ejecutarlo se deben crear dos tablas más. Estas son **historico_users** e **historico_loans**, que van a servir de apoyo para lo que debe ejecutar el disparador.

Este disparador salta antes de realizar el borrado en la tabla **users** y se realiza por cada fila.

Este consiste en crear una fila en la tabla **historico_users** que deje registrados aquellos usuarios que estuvieron registrados en la biblioteca pero que fueron eliminados de **users**. Al eliminar a esos usuarios, los préstamos asociados a estos también se borran de la tabla **loans** pero antes de esa acción son trasladados a la tabla **historico_loans** para dejar constancia de estos.

b) Código en SQL:

Tablas de apoyo:

```
CREATE TABLE historico_users (  
  USER_ID      CHAR(10),  
  ID_CARD      CHAR(17) NOT NULL,  
  NAME         VARCHAR2(80) NOT NULL,  
  SURNAME1     VARCHAR2(80) NOT NULL,  
  SURNAME2     VARCHAR2(80),  
  BIRTHDATE    DATE NOT NULL,  
  TOWN         VARCHAR2(50) NOT NULL,  
  PROVINCE     VARCHAR2(22) NOT NULL,  
  ADDRESS      VARCHAR2(150) NOT NULL,  
  EMAIL        VARCHAR2(100),  
  PHONE        NUMBER(9) NOT NULL,  
  TYPE         CHAR(1) NOT NULL,  
  BAN_UP2      DATE,  
  CONSTRAINT pk_hist_users PRIMARY KEY(user_id),  
  CONSTRAINT fk_hist_users_municipalities FOREIGN KEY(town,province)  
    REFERENCES municipalities(town,province),  
  CONSTRAINT ck_hist_usr_phone CHECK (phone>99999999)  
);
```

```
CREATE TABLE historico_loans (  
  SIGNATURE     CHAR(5),  
  USER_ID      CHAR(10),  
  STOPDATE     DATE,  
  TOWN         VARCHAR2(50) NOT NULL,
```

```
PROVINCE      VARCHAR2(22) NOT NULL,  
TYPE          CHAR(1) NOT NULL,  
TIME          NUMBER(5) default(0) NOT NULL,  
RETURN       DATE,  
CONSTRAINT pk_hist_loans PRIMARY KEY(signature,user_id,stopdate),  
CONSTRAINT fk_hist_loans_users FOREIGN KEY(user_id) REFERENCES historico_users  
(user_id),  
CONSTRAINT fk_hist_loans_copies FOREIGN KEY(signature) REFERENCES copies  
(signature),  
CONSTRAINT fk_hist_loans_services FOREIGN KEY(town,province,stopdate)  
REFERENCES services (town,province,taskdate),  
CONSTRAINT ck_hist_loans_dates CHECK (return is null or return>stopdate)  
);
```

Disparador:

```
CREATE OR REPLACE TRIGGER historicos  
BEFORE DELETE ON users  
FOR EACH ROW  
BEGIN  
    INSERT INTO historico_users values(:OLD.user_id, :OLD.id_card,  
:OLD.name, :OLD.surname1, :OLD.surname2, :OLD.birthdate, :OLD.town,  
:OLD.province, :OLD.address, :OLD.email, :OLD.phone, :OLD.type,  
:OLD.ban_up2);  
    INSERT INTO historico_loans (select distinct * from loans where  
user_id=:OLD.user_id);  
    DELETE FROM loans where loans.user_id=:OLD.user_id;  
END historicos;
```

c) Batería de pruebas:

- Borrar en users -> ver si se almacena en los históricos y si se ha borrado de loans

6. Conclusiones

Dentro de esta conclusión nos gustaría remarcar varias cosas:

- No se ha realizado el álgebra relacional en otros apartados porque hemos considerado que las consultas realizadas son bastante simples, por lo que no veíamos la necesidad de hacer álgebra relacional para estas.
- No se han podido realizar muchas pruebas por un problema al intentar borrar el user actual. Se intentó de innumerables formas borrar el user y desde distintos ordenadores. Sin embargo, siempre que se intentaba borrar, el programa se quedaba pillado. No sabemos qué era lo que podía fallar porque se quitaron todas las cosas que actuaban sobre esa tabla, pero fue imposible. Lamentamos enormemente no haber podido entregar todas las pruebas necesarias, aunque las dejamos indicadas en sus respectivos apartados.
- Lo que hemos interpretado y asumido en cada apartado está definido en cada uno de ellos.

Como conclusión de este trabajo, es cierto que pensábamos que el resultado sería mejor de lo acontecido. Sin embargo, estamos más o menos contentas porque, mientras lo hacíamos, probábamos las cosas en una pequeña base de datos que habíamos montado y en esta sí que funcionaban las pruebas de todo según lo esperado. Nos hubiera gustado entregar algo mejor, y aunque creemos que nuestro esfuerzo no se ve 100% representado, puesto que en este trabajo ha habido mucha organización y horas echadas, creemos que lo que hemos presentado es una clara mejora a lo anterior.